



UNIVERZITET U NOVOM SADU FAKULTET TEHNIČKIH NAUKA



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
NOVI SAD

ISPITNI RAD

Kandidat: Lazar Topić

Broj indeksa: SV62/2023

Predmet: Paralelno programiranje

Tema rada: Realizacija Hough-ove transformacije upotrebom OneAPI/TBB ili OpenCL okvira

Mentor rada: Pavle Vasiljević

Novi Sad, jun, 2026.

Sadržaj

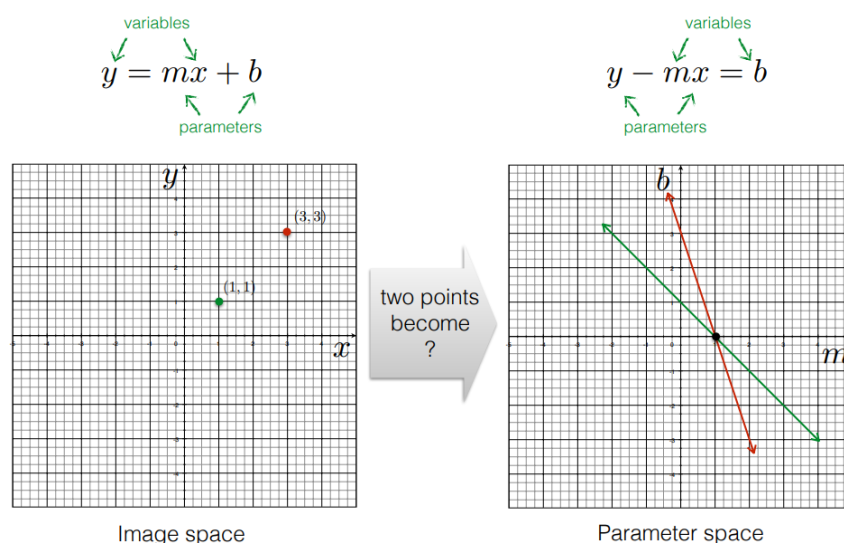
1. Uvod.....	1
1.1 Definicija Hough-ove transformacije	1
1.2 Cilj zadatka	4
1.3 Tehnologije	4
2. Analiza problema	5
2.1 Identifikacija faza	6
2.2 Usko grlo.....	6
2.3 Problem sinhronizacije	6
3. Implementacija	7
3.1 TBB Flow Graph i batch obrada	7
3.2 Faza 1 – Grayscale konverzija	8
3.3 Faza 2 – Detekcija ivica.....	8
3.3.1 Otsu prag.....	8
3.3.2 Sobel filter	8
3.4 Faza 3 – Izgradnja Hough akumulatora.....	9
3.5 Faza 4 – Detekcija linija i generisanje rezultata.....	10
3.5.1 Non-Maximum Supression.....	10
3.5.2 Crtanje linija i čuvanje rezultata	10
4. Testiranje i rezultati	11
4.1 Testni skupovi	11
4.2 Rezultati vizuelizacije.....	12
4.2.1 Detektovane ivice	12
4.2.2 Hough akumulator.....	13
4.2.3 Finalni rezultat	14
4.3 Analiza performansi	15
5. Zaključak	16

1. Uvod

1.1 Definicija Hough-ove transformacije

Hough transformacija je tehnika iz oblasti obrade digitalnih slika koja se koristi za prepoznavanje geometrijskih objekata unutar slike, najčešće pravih linija, ali i kružnica, elipsa ili proizvoljnih oblika definisanih parametara. Suština metode je da se problem prepoznavanja oblika svede na problem glasanja: svaka tačka koja pripada nekoj ivici slike “glasa” za sve moguće oblike koji bi mogli prolaziti kroz tu tačku, a oblici koji dobiju najviše glasova proglašavaju se za detektovane.

Klasičan pristup detekciji linija bio bi da se prava predstavi u obliku $y = mx + b$, gde su m i b parametri prave. Problem ovog zapisa je što vertikalne linija imaju nagib m koji teži ka bekonačnosti, što parametarski prostor čini neograničenim i praktično neupotrebljivim za implementaciju preko niza (akumulatora) konačne veličine.



Slika 1: Primer prostora slike i parametarskog prostora gde je prava oblika $y = mx + b$

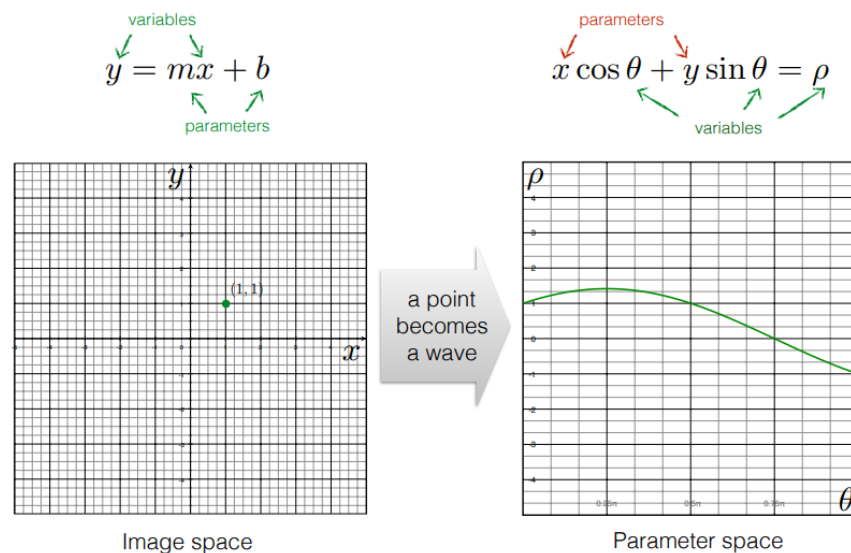
How big does the accumulator need to be?


$$-\infty \leq m \leq \infty$$

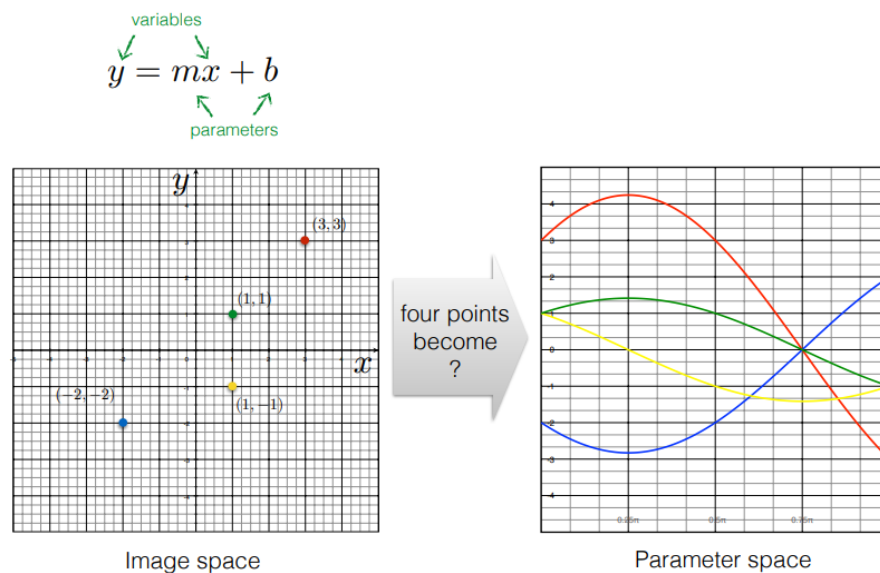
Iz tog razloga, danas se umesto (m, b) zapisa koristi Heseov normalni oblik prave:

gde ρ predstavlja najkraće rastojanje od koordinatnog početka do prave, a θ je ugao koji normala na pravu zaklapa sa x-osom. Na ovaj način se svaka prava u (x, y) prostoru slike preslikava u jednu tačku u (ρ, θ) parametarskom prostoru, dok se, obrnuto jedna tačka (x, y) iz slike preslikava u sinusoidu u (ρ, θ) prostoru, jer postoji beskonačno mnogo pravih koje prolaze kroz tu tačku, svaka pod drugim uglom θ i sa drugim rastojanjem ρ .

2



Slika 3: Preslikavanje tačke u prostoru slike, na sinusoidu u parametarskom prostoru



Slika 4: Primer kada više tačaka leži na istoj pravoj se u parametarskom prostoru seku u istoj tački

Algoritam se u praksi implementira preko akumulatora, dvodimenzionalnog niza čije ose predstavljaju kvantizovane vrednosti ρ i θ . Za svaku ivičnu tačku (x, y) na slici, algoritam prolazi kroz sve moguće vrednosti ugla θ , izračunava odgovarajuće ρ prema formuli, i uvećava (glasa) odgovarajuću ćeliju akumulatora za jedan. Nakon što se ovaj postupak izvrši za sve ivične tačke, ćelija akumulatora sa najvećim brojem glasova (lokalni maksimumi) odgovaraju pravim linijama koje su najzastupljenije na slici.

1.2 Cilj zadatka

Cilj ovog rada je implementacija kompletnog pipeline-a za detekciju pravih linija na digitalnoj slici primenom Hough transformacije, sa fokusom na paralelizaciju obrade pomoću Intel oneAPI TBB biblioteke.

Ideja je da se čitava funkcionalnost podeli na faze koje mogu da se izvršavaju paralelno na više jezgara procesora, čime se smanjuje vreme obrade.

1.3 Tehnologije

Za implementaciju je korišćen programski jezik C++. Za paralelizaciju je korišćena Intel oneAPI TBB biblioteka, kao skup C++ template-a koje olakšavaju pisanje paralelnog koda. U radu su korišćeni TBB Graf toka (za organizaciju faza obrade kao grafa zavinsosti) i paralelni algoritmi `parallel_for` i `parallel_reduce`, kao i konkurentni kontejner `concurrent_vector` i atomske operacije za bezbedan pristup deljenim podacima iz više niti.

Za učitavanje i čuvanje slika poslužio sam se STB Image bibliotekom. To je header-only biblioteka koja omogućava rad sa različitim formatima slika. Preuzeo sam .h fajlove sa sledećeg linka: <https://github.com/nothings/stb>

2. Analiza problema

Ako se zanemari algoritamski deo zadatka, glavni problem ostaje paralelizacija sekvencijalnog koda i verifikacija oba programa.

Priroda paralelizma ovog programa leži u samoj obradi slike, skoro svaka operacija u pipeline-u (konverzija u sive tonove, Sobel filter, izgradnja akumulatora) primenjuje istu računsku operaciju na veliki broj piksela, pri čemu je rezultat za jedan piksel najčešće nezavisan od rezultata za druge piksele.

S druge strane, ne paralelizujemo sve faze na isti način. Neke faze su trivijalno paralelne, na primer svaki piksel se obrađuje potpuno samoostalno, neke faze zahtevaju da se rezultati više niti spoje u jednu vrednost, kao što je slučaj kod izračunavanja Otsu praga, dok kod izgradnje Hough akumulatora imamo i potencijalan problem trke do podataka.

Dakle ideja ove razlike paralelizma između faza zahtevao je ne samo implementaciju `parallel_for` petlje, već korišćenje `parallel_reduce`, atomskih operacija i konkurentnih kontejnera.

2.1 Identifikacija faza

U implementaciji je obrada slika podeljena na četiri faze, organizovane kao čvorovi grafa toka podataka.

Faza 1 – Grayscale konverzija: svaki piksel originalne RGB slike se pretvara u jednu vrednost intenziteta primenom formule $Y = 0.299R + 0.587G + 0.114B$.

Faza 2 – Detekcija ivica: na sliku sa sivim tonovima primenjuju se Sobel kerneli kako bi se dobio gradijent intenziteta u svakoj tački, a zatim se Otsu metodom računa prag binarizacije na osnovu histograma slike.

Faza 3 – Izgradnja Hough akumulatora: za svaku tačku koja je u prethodnoj fazi prepoznata kao ivica, računaju se vrednosti ρ za sve uglove θ i odgovarajuće ćelije akumulatora se uvećavaju.

Faza 4 – Detekcija linija i čuvanje rezultata: pretraga akumulatora za lokalnim maksimumima, crtanje pronadjenih linija na originalnoj slici i upis svih izlaznih fajlova.

2.2 Usko grlo

Od sve četiri faze, izgradnja Hough akumulatora je vremenski najzahtevnija, i to iz razloga što za razliku od Grayscale i Sobel faza, gde se na svaki piksel primenjuje malo broj operacija, u Fazi 3 se za svaki ivični piksel ponavlja čitava petlja po uglu θ . U implementaciji je broj kvantizovanih uglova NUM_THETA postavljen na 180, što znači da se za samo jednu ivičnu tačku računa 180 vrednosti sinusa, 180 vrednosti kosinusa, i 180 sabiranja/upisa u akumulator.

Sa stanovišta paralelizacije, dobro je da su sve te operacije i dalje nezavisne po ivičnim tačkama. Loša vest je da svi rezultati na kraju „sleću“ u istu deljenu strukturu, a to je akumulator, što nas vodi do problema sinhronizacije.

2.3 Problem sinhronizacije

Akumulator predstavlja dvodimenzioni niz brojača, jedna ćelija po paru (ρ, θ) . Paralelizacijom Faze 3, više niti istovremeno obrađuje različite ivične piksele, ali je sasvim moguće da dve ili više njih u istom trenutku računaju par (ρ, θ) koji upada u istu ćeliju akumulatora i pokušavaju da je uvećaju za 1. Ako bi se uvećanje izvelo kao obična operacija došlo bi do trke do podataka. U implementaciji sam ovo rešio korišćenjem atomskih instrukcija, o kojima će biti više reči kasnije.

3. Implementacija

Implementacija je organizovana kao pipeline od četiri faze, pri čemu svaka faza obradjuje jednu sliku i prosledjuje rezultat sledećoj. Čitava rhitektura počiva na Intel TBB Flow Graph-u, koji definiše redosled faza i, u paralelnom modu, omogućava da se više slika obradjuje istovremeno kroz isti graf. U nastavku je svaka faza opisana detaljno.

3.1 TBB Flow Graph i batch obrada

Tbb::flow::graph je objekat koji opisuje tok podataka izmedju čvorova (function_node). Svaki function_node prima neki tip podataka, obavlja posao i vraća rezultat koji automatski ide ka sledećem čvoru.

Graf ima četiri čvora (phase1/2/3/4) koji su sekvencijalno povezani pozivima make_edge. Svaki čvor je kreiran sa konkurentnošću unlimited, što znači da TBB može pokrenuti više instanci istog čvora u paraleli, na primer dok jedna slika prolazi kroz Fazu 2, druga može početi Fazu 1 za sledeću sliku.

Sve podatke koji se prenose izmedju faza drži struktura PipelineData, koja sadrži učitane originalnu sliku, medjurezultate i konfiguracione parametre. Ova struktura se ne kopira, izmedju čvorova se prenosi kao std::shared_ptr<>. Ovo garantuje da struktura ostaje živa dok je bar jedan čvor koristi, i automatski se odlobadja kada prodje ceo pipeline.

Batch obrada je implementirana u funkciji executeBatch. U sekvencijalnom modu svaka slika se ubaci u graf (try_put) i program odmah čeka da prodje kroz ceo pipeline (g.wait_for_all()) pre nego što ubaci sledeću. U paralelnom modu, sve slike se ubacuju u graf jedna za drugom bez čekanja, a g.wait_for_all() se poziva tek na kraju. Sama funkcija runPipeline poziva executeBatch dva puta, prvi put u sekvencijalnom, drugi u paralelnom modu, gde se mere vremena izvršavanja i na kraju njihovo poredjenje upisuje u izveštaj.

3.2 Faza 1 – Grayscale konverzija

Hough transformacija radi sa intenzitetima piksela, a ne sa bojama, pa je prvi korak prevodjenje RGB slike u sliku sa sivim tonovima. Za svaki piksel se primenjuje formula:

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

Rezultat je nova slika sa jednim kanalom, gde vrednost svakog piksela predstavlja njegovu percipiranu osetljivost.

Paralelizacija je izvedena sa idejom da vrednost svakog piksela zavisi isključivo od spostvenih R,G,B vrednosti, bez ikakve veze sa susednim pikselima. Implementacija koristi `tbb::parallel_for` sa `blocked_range<int>` po redovima slike. TBB deli opseg redova (0 do height) na blokove i raspodeljuje ih dostupnim nitima. U sekvencijalnom modu isti lambda se izvršava sekvencijalno u jednom pozivu, bez izmene logike, jedina razlika je u tome da li se poziva `parallel_for` ili se lambda poziva direktno sa punim opsegom.

3.3 Faza 2 – Detekcija ivica

3.3.1 Otsu prag

Pre Sobel filtera računa se prag binarizacije Otsu metodom. Otsu algoritam analizira histogram slike (distribuciju vrednosti intenziteta od 0 do 255) i pronalazi prag koji maksimizuje varijansu izmedju dva dela, tamnog i svetlog. Prednost metode je što prag nije fiksni broj koji ručno podešavamo, nego se automatski adaptira na sadržaj svake slike.

Izgradnja histograma zahteva prolaz kroz sve piksele i sabiranje, za šta je u implementaciji korišćen `parallel_reduce`. Svaka nit dobija podopseg piksela, gradi sopstveni lokalni histogram, a na kraju se lokalni histogrami spajaju u jedno sabiranje po svakom od 256 binova.

Dobijeni Otsu prag se pomnoži dodatno sa faktorom 0.85 pre nego što se preda Sobel filteru. Snižavanje praga čini detektor osetljivijim na slabije ivice.

3.3.2 Sobel filter

Sobel filter detektuje ivice računanjem gradijenta intenziteta u svakoj tački slike.

Koriste se dva 3x3 kernela. G_x (horizontalni gradijent) i G_y (vertikalni gradijent).

$$G_x: [-1, 0, +1; -2, 0, +2; -1, 0, +1]$$

$$G_y: [-1, -2, -1; 0, 0, 0; +1, +2, +1]$$

Za svaki unutrašnji piksel uzimaju se vrednosti 8 suseda, primenjuju se oba kernela konvolucijom, i računa magnituda gradijenta. Ako je magnituda veća od praga dobijenog iz Otsu metode, piksel se označi kao ivica (vrednost 255), u suprotnom ostaje 0.

Paralelizovana je, jer vrednost gradijenta za jedan piksel zavisi samo od njegovih 8 neposrednih suseda, koji se samo čitaju, pa ne postoji nikakav konflikt između niti. Implementacija koristi `parallel_for`.

3.4 Faza 3 – Izgradnja Hough akumulatora

Za svaki piksel koji je u prethodnoj fazi označen kao ivica, prolazi se kroz sve kvantizovane uglove od θ do π i za svaki θ računa se vrednost ρ prema Hough formuli:

$$\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta)$$

Vrednost ρ se mapira u indeks ćelije akumulatora. Pošto ρ može biti i negativno (za uglove gde kosinus ili sinus daju negativnu vrednost za date x , y koordinate piksela), opseg ρ se kreće od $-\rho_{\text{Max}}$ do $+\rho_{\text{Max}}$, gde je ρ_{Max} dijagonala slike. Indeksiranje u akumulatoru (dimenzija $\text{numRho} \times 180$) je:

$$\text{rhoIdx} = ((\rho + \rho_{\text{Max}}) / (2 \cdot \rho_{\text{Max}})) \cdot \text{numRho}$$

Radi optimizacije, $\sin$ i $\cos$ vrednosti za svih 180 uglova se unapred izračunavaju i čuvaju u `sinTable` i `cosTable`.

Problem koji je specifičan za ovu fazu je thread safetiz akumulatora. Svaka nit obrađuje različite redove slike, ali se rezultati glasanja preklapaju, na primer dve različite ivične tačke iz različitih redova mogu lako generisati isti par (rhoIdx, t) i pokušati da uvećaju istu ćeliju akumulatora. Rešenje je deklaracija akumulatora kao `std::vector<std::vector<std::atomic<int>>>`.

Svaka ćelija je atomski integer i koristi se:

$$\text{accumulator}[\text{rhoIdx}][t].\text{fetch_add}(1)$$

`fetch_add` je podržana atomska instrukcija, procesor garantuje da će se čitanje, uvećavanje i upisivanje biti izvedeni kao jedna neprekinuta operacija, bez mogućnosti da druga nit upadne između.

3.5 Faza 4 – Detekcija linija i generisanje rezultata

3.5.1 Non-Maximum Supression

Nakon izgradnje akumulatora, svaka ćelija sa dovoljno glasova potencijalno predstavlja jednu pravu liniju. Međutim, jedna realni linija na slici obično ne odgovara jednoj tački u akumulatoru, nego grupi susednih ćelija sa visokim brojem glasova, zbog šuma i kvantizacije. Bez filtera, program bi za jednu pravu liniju na slici vratio desetine gotovo identičnih detektovanih linija.

Non-Maxim Supression rešava ovaj problem: ćelija se proglašava detektovanom linijom akp ispunjava dva uslova: ima više glasova od praga ($\text{HOUGH_THRESHOLD_RATIO} = 0.32$, dakle mora imati bar 32% od maksimalnog broja glasova u akumulatoru), i ima više glasova od svih svojih suseda unutar prozora veličine $\text{neighborhoodSize} * \text{neighborhoodSize}$. Time se za svaku grupu susednih jakih ćelija zadržava ona sa najviše glasova, koja najpreciznije opisuje pravu.

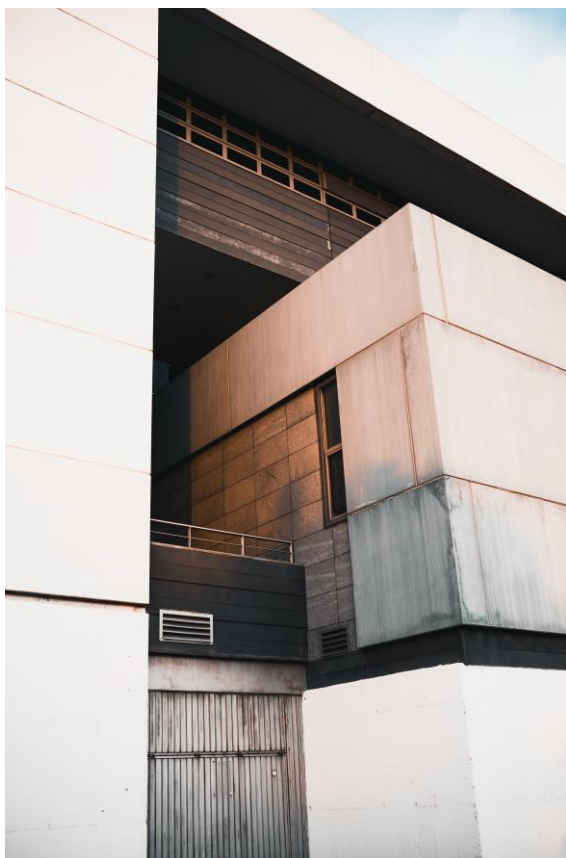
3.5.2 Crtanje linija i čuvanje rezultata

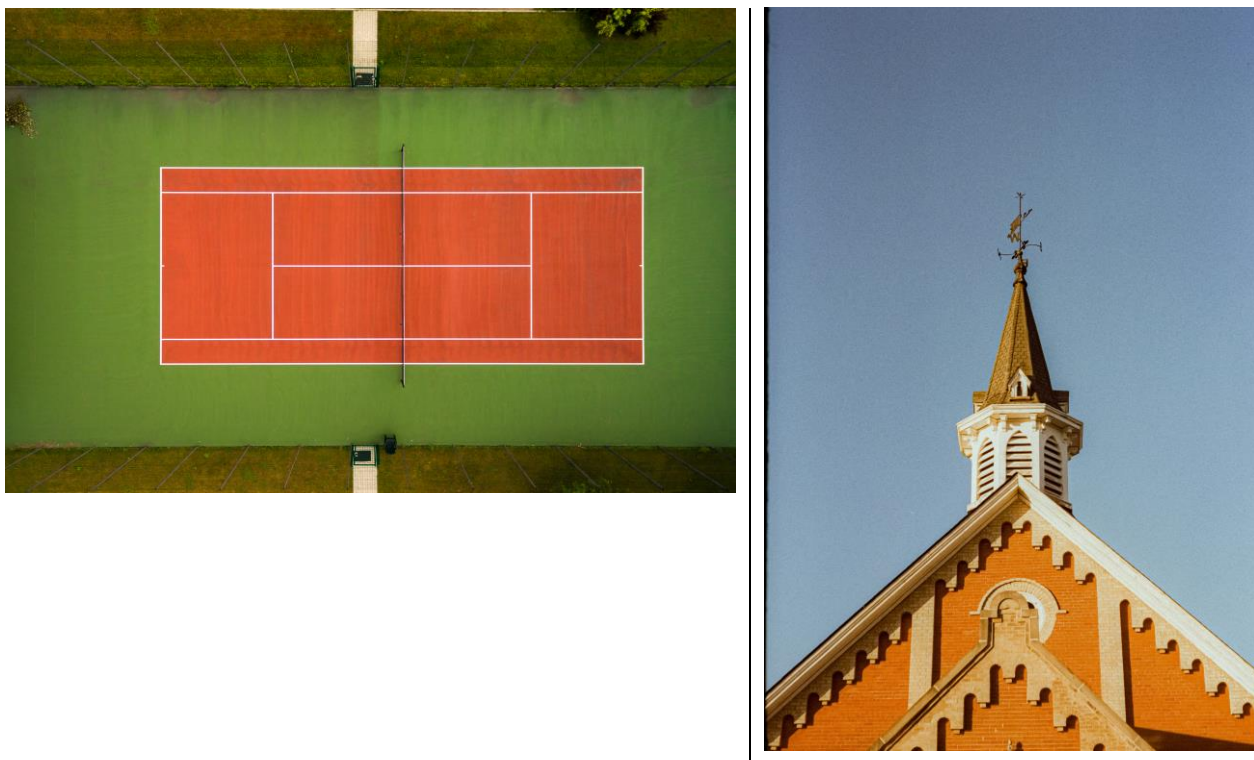
Svaka detektovana linija opisana je parom (ρ, θ) . Da bi se nacrtala na slici, potrebno je izračunati tačku na liniji najbližu koordinatnom početku ($x_0 = \rho \cdot \cos\theta$, $y_0 = \rho \cdot \sin\theta$), a zatim tu tačku produžiti u oba smera za veliki L . Crtanje se vrši DDA (Digital Differential Analyzer) algoritmom, inkrementalno hodanje po liniji piksel po piksel bojenjem svake tačke u crvenu boju.

4. Testiranje i rezultati

4.1 Testni skupovi

Za testiranje su korišćene četiri slike, sve u PNG formatu i učitavaju se kao RGB.





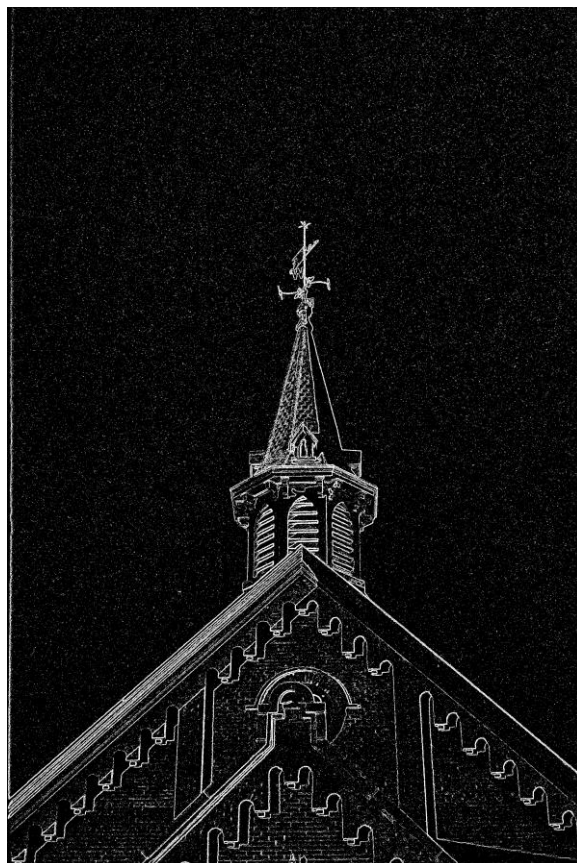
Slika 5. Četiri slike korišćene kao test slučajevi ovog programa

4.2 Rezultati vizuelizacije

Za svaku testnu sliku program generiše tri izlazna fajla koji vizualizuju svaki korak pipeline-a: detektovane ivice, Hough akumulator i finalni rezultat sa nacrtanim linijama. U nastavku je prikazana realizacija celog postupka za jednu od slika.

4.2.1 Detektovane ivice

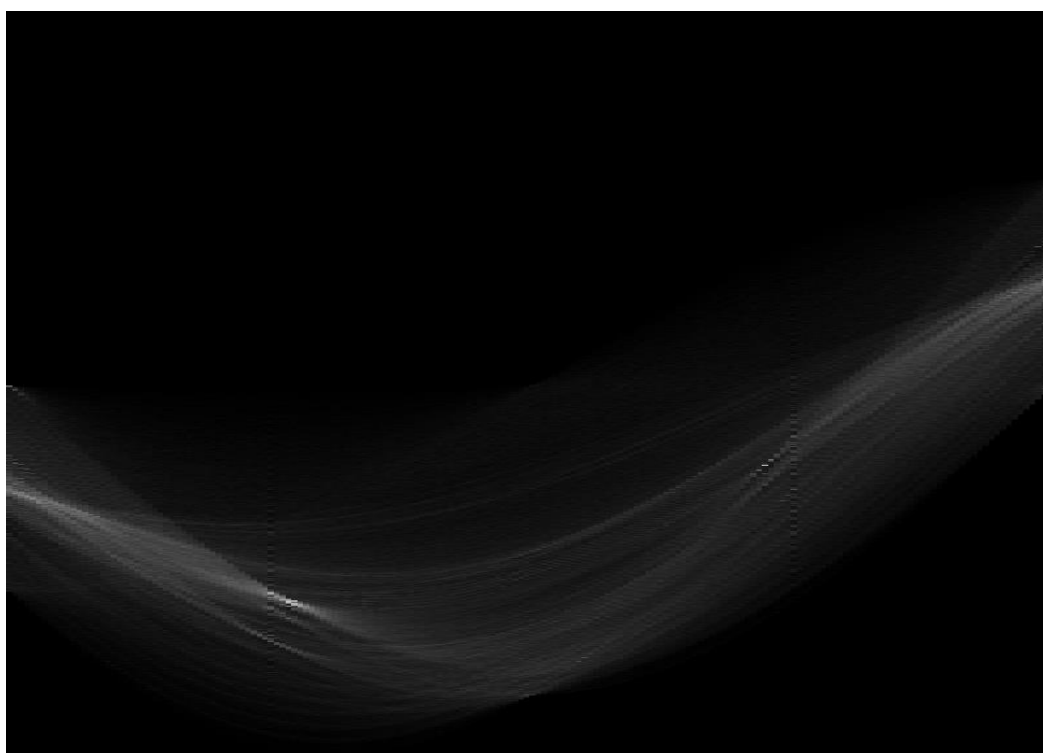
Izlaz Faze 2 je binarna slika u sivim tonovima: pikseli prepoznati kao ivice su beli, a ostali crni. Što su ivice oštrije i kontrasniji originalni sadržaj, to je ovaj prikaz čistiji i bez šuma.



Slika 6. Izlaz Faze 2, nakon detekcije ivica

4.2.2 Hough akumulator

Akumulator je vizualizovan kao 800x800 slika u sivim tonovima: horizontalna osa odgovara uglu θ , vertikalna osa vrednosti ρ . Svetlija tačka znači više glasova, tj više ivičnih piksela koji glasaju za tu kombinaciju (ρ, θ) . Svaka svetla tačka u akumulatoru odgovara jednoj detektovanoj pravoj liniji na originalnoj slici, upravo one tačke koje NMS označi kao lokalne maksimume iznad praga bivaju nacrtane na finalnoj slici. Prepoznatljiv vizuelni obrazac su sinusoide koje se ukrštaju.



Slika 7. Vizualizacija akumulatora kao ukrštanje sinusa

4.2.3 Finalni rezultat

Finalni izlaz je originalna slika na kojoj su crvenom bojom nacrtane sve detektovane prave linije.



Slika 8. Finalni rezultat Hough transformacije na slici krova

4.3 Analiza performansi

Performanse su merene pokretanjem kompletnog batch-a oba puta, jednom u sekvencijalnom, jednom u paralelnom modu i merenjem ukupnog vremena.

Ubrzanje od 2.81x na batch-u od četiri velike slike pokazuje da paralelizacije donosi benefit na zahtevnim ulazima. Međutim treba imati na umu da ovo nije čisto ubrzanje samo algoritamskih faza, ukupno mereno vreme uključuje i učitavanje slika i pisanje izlaza na disk, što su sekvencijalne I/O operacije koje se ne ubrzavaju bez obzira na broj jezgara.

5. Zaključak

Napravljen je jedan koncept za realizaciju datog problema. Napravljen je i verifikovan sekvencijalni i paralelni model koji obuhvata problem opisan u zadatku.

Serijski i paralelni program su pokretani za iste testne slike na istom računaru u cilju merenja vremena i ubrzanja. Korišćenjem TBB Flow Graph arhitekture, proces je organizovan kao asinhroni pipeline koji omogućava istovremenu obradu više slika, dok su unutar faza primenjeni algoritmi paralelnog programiranja, uz korišćenje konkurentnih kontejnera.

Upotreba TBB biblioteke podigla je nivo apstrakcije, omogućivši razvoj paralelnog koda bez ručnog upravljanja nitima i sinhronizacijom.